Competitive Programming

Section 44 Editorial: String Algorithms II

Competitive Programming Course

2026-10-20

Table of contents

1 Editorial overview **3**
- 1.1 Problem-specific proof map . . . 3
- 1.2 Detailed solution guide . . . 3

2 A. Trie Prefix Count **5**
- 2.1 Restatement . . . 5
- 2.2 Observations and algorithm . . . 5
- 2.3 Correctness proof . . . 5
- 2.4 Complexity . . . 5
- 2.5 Tests that matter . . . 5

3 B. Palindrome Queries **6**
- 3.1 Restatement . . . 6
- 3.2 Observations and algorithm . . . 6
- 3.3 Correctness proof . . . 6
- 3.4 Complexity . . . 6
- 3.5 Tests that matter . . . 6

4 C. Distinct Substrings **7**
- 4.1 Restatement . . . 7
- 4.2 Observations and algorithm . . . 7
- 4.3 Correctness proof . . . 8
- 4.4 Complexity . . . 8
- 4.5 Tests that matter . . . 8

5 D. Unique Prefix Lengths **9**
- 5.1 Restatement . . . 9
- 5.2 Constraints and traps . . . 9
- 5.3 Observations and trace . . . 9
- 5.4 Algorithm . . . 9
- 5.5 Correctness proof . . . 10
- 5.6 Complexity . . . 10
- 5.7 C++ implementation notes . . . 10
- 5.8 Python implementation notes . . . 10
- 5.9 Common wrong answers . . . 10
- 5.10 Tests that matter . . . 11

6 What to carry forward **12**

7 Testing discussion **13**

1 Editorial overview

1.1 Problem-specific proof map

A. Trie Prefix Count. Incrementing every trie node visited by an inserted word makes that node's count equal to the number of words sharing its prefix. Following a query prefix therefore returns the desired count in its length.

B. Palindrome Queries. The local constraints permit direct mirrored comparison: a substring is a palindrome exactly when every symmetric pair is equal. This is `O(length)` per query and avoids probabilistic hash collisions.

C. Distinct Substrings. The local task uses a suffix automaton. State `v` groups several substrings with the same ending positions. Its represented lengths run from `len[link[v]] + 1` through `len[v]`, one substring per length, so it contributes `len[v] - len[link[v]]` distinct substrings. For `aba`, the three non-root states represent `a`, then `b, ab`, then `ba, aba`: the answer is `1 + 2 + 2 = 5`. The derivation is detailed in problem C below.

1.2 Detailed solution guide

A. Trie Prefix Count. Each trie node represents exactly one prefix: the string on the path from the root to that node. During insertion, increment `cnt` at the root and after moving to each next node. Therefore, after all insertions, `cnt[v]` is the number of dictionary words whose path passes through `v`, which is precisely the number of words with that prefix. A query walks the prefix characters; a missing edge means zero matches.

B. Palindrome Queries. For the local constraints, comparing mirrored characters is the clearest exact solution. For substring `[l,r]`, compare `s[l]` with `s[r]`, then `s[l+1]` with `s[r-1]`, and so on. If all pairs match, the substring equals its reverse. If one pair differs, the two strings differ at a mirrored position, so it is not a palindrome. In a larger contest version, the same condition can be checked with forward and reverse rolling hashes.

C. Distinct Substrings. The local solution is a suffix automaton. Each state contributes `len[v] - len[link[v]]` new substrings: these are exactly the lengths whose end-position class first appears at that state. Summing over non-root states gives the answer. Enumerating every substring into a set is correct only for tiny random tests used to validate the automaton.

Common pitfalls. Trie counters count prefixes, not terminal words only. Palindrome queries must convert input indices consistently. For distinct substrings, enumeration is pedagogical and exact at small limits, but a suffix automaton is required for large external tasks.

This section shows three string representations.

- A uses a trie. Each inserted word increments counters on the nodes along its path, so the counter at the prefix endpoint is exactly the number of words with that prefix.
- B checks palindromes by comparing mirrored characters. For larger limits this can be replaced by forward/reverse hashes, but the local proof is exact.

- C counts distinct substrings with a suffix automaton. The local invariant is the contribution `len[v] - len[link[v]]` per non-root state; enumeration is reserved for tiny brute-force tests.

The lesson exposes each structure's operations independently of exercise input and output. Brute force is used as a testing oracle.

The reusable C++ and Python implementations are in the lesson: trie operations in Chapter 2, rolling hash in Chapter 3, and suffix automaton in Chapter 4. This editorial focuses on the exercise-specific reasoning and proofs.

2 A. Trie Prefix Count

2.1 Restatement

Given dictionary words and prefix queries, count words having each prefix.

2.2 Observations and algorithm

Each trie node is one prefix. Increment its counter for every inserted word visiting it; walk a query to its endpoint and return that counter, or zero on a missing edge. Inserting `apple, ape, bat` makes query `ap` return two.

2.3 Correctness proof

After every insertion, a node counter equals the words with that node's prefix, so the query endpoint is exactly the requested count. QED.

2.4 Complexity

Work is linear in total inserted and queried characters; memory is one node per distinct prefix.

2.5 Tests that matter

Test duplicate words, one word that prefixes another, an absent prefix, and a one-character query.

3 B. Palindrome Queries

The trie problem organized many dictionary words by their shared prefixes. Here the input is one fixed text, and each query selects a range inside it. The question is about the order of the selected characters: do we obtain the same string when reading them backward?

We turn that definition into a direct comparison algorithm. Chapter 3 of the lesson develops rolling hashes for larger query loads and provides their reusable implementations.

3.1 Restatement

A palindrome reads the same in both directions, such as `aca` or `abba`. For each selected range, determine whether its characters form a palindrome. The text stays unchanged.

3.2 Observations and algorithm

The local bounds permit exact mirrored comparison. Compare the first and last characters of the selected part, then move inward and stop on a mismatch. In `abacaba`, positions 2 through 4 select `bac` (NO), while positions 3 through 5 select `aca` (YES).

3.3 Correctness proof

A substring equals its reverse iff each mirrored character pair is equal, which is exactly the two-pointer condition. QED.

3.4 Complexity

Each query is `O(r-l+1)` time and `O(1)` extra memory; this exact approach is only suitable for the local small-query contract. An inward two-pointer scan avoids constructing substring copies. The reasoning depends only on comparing symmetric characters, regardless of how the positions are numbered.

3.5 Tests that matter

Test singleton, full string, even and odd lengths, and a mismatch in the outer pair.

4 C. Distinct Substrings

4.1 Restatement

Print the number of distinct non-empty substrings of a string.

4.2 Observations and algorithm

Build a suffix automaton, then count the distinct substrings represented by each non-root state. A state can represent several strings, so counting states alone is insufficient.

4.2.1 What a state represents

For a substring, its ending positions are all positions where an occurrence finishes. Substrings with exactly the same set of ending positions share a state. In `aba`, with positions numbered from 1:

- `a` ends at positions 1 and 3.
- `b` and `ab` end only at position 2, so they share a state.
- `ba` and `aba` end only at position 3, so they share a state.

These are groups of distinct string values. Multiple occurrences of the same string do not create extra entries in the answer.

4.2.2 Why subtract the two lengths?

`len[v]` is the longest length in state `v`. The suffix link `link[v]` points to the state of the longest suffix that belongs to a different group. The shortest length still in `v` is therefore `len[link[v]] + 1`.

State `v` contains exactly one substring for each integer length in this range:

```
len[link[v]] + 1, ..., len[v]
```

For a fixed length, two strings with the same ending positions must be equal: choose one ending position, and the length fixes all characters of the string. The represented lengths form a consecutive range; taking shorter suffixes stays in this group until the suffix-link boundary is reached.

The number of integers in an inclusive range from `a + 1` to `b` is `b - (a + 1) + 1 = b - a`. Hence the contribution is:

```
len[v] - len[link[v]]
```

For `aba`, the contributions are:

- State for `a`: `len[v] = 1`, link to the root of length 0. Contribution: `1 - 0 = 1`, for `a`.
- State for `b, ab`: `len[v] = 2`, link to the root of length 0. Contribution: `2 - 0 = 2`, for lengths 1 and 2.
- State for `ba, aba`: `len[v] = 3`, link to the `a` state of length 1. Contribution: `3 - 1 = 2`, for lengths 2 and 3.

The answer is `1 + 2 + 2 = 5`: `a`, `b`, `ab`, `ba`, `aba`. Length 1 appears in two different states, for `a` and `b`; this is correct because they are different strings. The length intervals need not be disjoint across states. The groups of strings are disjoint.

Finally, sum the contributions over every non-root state. Exclude the root because it represents the empty string. For `ababa`, the same calculation gives 9 distinct nonempty substrings.

4.3 Correctness proof

Every distinct nonempty substring has exactly one ending-position set, so it belongs to exactly one non-root state. Within state `v`, the consecutive lengths from `len[link[v]] + 1` through `len[v]` each identify exactly one substring. Thus `len[v] - len[link[v]]` counts precisely that state's strings. Summing these disjoint groups counts each distinct substring once. Clones preserve the state groups during construction. QED.

4.4 Complexity

There is one root and at most `2*n-1` states for `n >= 2`; a one-character string has two states. Memory is `O(n)` for a fixed alphabet. Construction with C++ maps takes `O(n log alphabet)` time; Python dictionaries give expected `O(n)` time for the fixed lowercase alphabet. Counting the final contributions is `O(n)` in both languages.

4.5 Tests that matter

Test `a`, `aaa`, all distinct characters, and an alternating string.

5 D. Unique Prefix Lengths

5.1 Restatement

For every input word, find the shortest nonempty prefix shared by exactly one input occurrence. Equal words are still separate occurrences, so no prefix can distinguish one copy from another. A word that is a prefix of a longer word may also have no answer.

This is the blind trie exercise for the section: the task asks for a property of all prefixes of each word, not merely a count for one supplied prefix.

5.2 Constraints and traps

There are up to `200,000` words and the same bound on total characters. Testing every prefix against every word is quadratic. The answer is an input-order answer, so duplicate words need repeated `-1` lines rather than being merged.

Do not mistake "the word ends here" for "the prefix is unique." For `do` and `dog`, the prefix `do` is a prefix of both input words even though only one word ends at that trie node.

5.3 Observations and trace

In a trie, store `pass[node]`: the number of input words whose path visits the node. During insertion, increment it after every character. Then a prefix is unique exactly when the node reached by that prefix has `pass == 1`.

Insert the sample's `cat`, `car`, `dog`, `do`, `cat`. The node for `ca` has `pass=3` because it serves both copies of `cat` and `car`. The node for `car` has `pass=1`, so `car` needs length three. The node for `do` has `pass=2`, and there is no later node for the shorter word `do`, so its answer is `-1`.

5.4 Algorithm

```
make a trie whose nodes store child links and pass counts
for every word:
    walk/create its character path
    increment pass at each reached node

for every word in input order:
    walk its trie path from the root
    print the first position whose node has pass = 1
    print -1 if no such position occurs
```

5.5 Correctness proof

Lemma. For every trie node representing prefix `p`, `pass[node]` equals the number of input words beginning with `p`.

Proof. A word visits exactly the nodes for its nonempty prefixes during insertion. The algorithm increments this node once for that word and never for a word that does not have the prefix. Summing over all input occurrences gives the stated count. □

Theorem. The algorithm prints the required shortest unique-prefix length for every word.

Proof. By the lemma, a scanned prefix is unique exactly when its node has `pass=1`. The second traversal checks prefixes in increasing length order, so the first such node is the shortest valid prefix. If none is found, every nonempty prefix is shared by at least two occurrences and `-1` is correct. □

5.6 Complexity

Let `L` be the total input length.

```
Time:   O(L)
Memory: O(L)
```

5.7 C++ implementation notes

Reuse the trie operations presented in Chapter 2 of the lesson. The problem-specific traversal is left to implement from the algorithm above. Maintain the distinction between node indices, prefix lengths, and word-ending markers. The root is never a candidate answer because the required prefix must be nonempty. Duplicate words each contribute to `pass` and therefore cannot be distinguished.

5.8 Python implementation notes

Use the same two-pass organization: insert the entire dictionary before searching for unique prefixes. Scanning each word's path once costs `O(L)` in total. Each new trie node must have its own child dictionary.

5.9 Common wrong answers

```
Counting only words ending at a node instead of every word passing through it.
Deduplicating equal strings before answering.
Returning a prefix for do when dog is also present.
Using the root count as an empty-prefix answer.
Scanning all words for every candidate prefix.
```

5.10 Tests that matter

```
Identical words: every copy must print -1.
A word that is a prefix of another word.
Words sharing a long prefix before branching.
A one-word input: answer is 1 for a nonempty word.
200,000 identical one-letter words: checks linear memory and output.
```

6 What to carry forward

Advanced algorithms become manageable when you state the invariant first, then code only the operations needed to preserve it.

7 Testing discussion

Brute-force small trie prefixes by `word.startswith(prefix)`, palindrome queries by reverse comparison, and distinct substrings with a set of all slices. Then run:

```
CP_TARGET=solution python3 sections/44_string_algorithms_ii/check.py
```